

ASSIGNMENT 1: FUNCTIONAL PROGRAMMING

The goal of this assignment is to implement interpreters for a minimalistic imperative language that we will call IMP.

- The runtime semantics of the IMP language assumes a **memory**, which assigns variable names to integers.
- A **variable name** is just a string.
- Programs in the IMP language are built from *expressions* which represent values and may depend on variables. For example, “`3 * x + 1`” is an expression representing a value, which depends on a variable `x`.
- Programs are also built from *statements* that represent data and control flow. For example, “`x := x + 1`” is a statement; more precisely, it is an assignment representing the operation that increases the value of the variable `x` by 1.
- A **value** in IMP may be an integer or a boolean.
- An **expression** may be:
 - A **variable**, such as “`n`”, “`amount`” or “`height`”.
 - A **constant**, which may be any value, including integers and booleans.
 - A **binary operation**:
 - * Sum (e.g. `e1 + e2`) — takes two integers and returns an integer.
 - * Product (e.g. `e1 * e2`) — takes two integers and returns an integer.
 - * And (e.g. `e1 && e2`) — takes two booleans and returns a boolean.
 - * Or (e.g. `e1 || e2`) — takes two booleans and returns a boolean.
 - * Equal (e.g. `e1 == e2`) — takes two integers and returns a boolean.
 - * Less than (e.g. `e1 < e2`) — takes two integers and returns a boolean.
 - A **unary operation**:
 - * Unary negation (e.g. `!e`) — takes a boolean and returns a boolean.
 - * Unary minus (e.g. `-e`) — takes an integer and returns an integer.
- A **statement** may be:
 - A **skip** statement that does not do anything.
 - An **assignment** of the form `x := expr`, where `x` is a variable and `expr` is an expression.
 - A **sequence** of the form `s1; s2`, where `s1` and `s2` are statements. The operational meaning of a sequence is: first execute `s1`, then execute `s2`.
 - An **if** construct of the form `if (e) {s1} else {s2}`. The operational meaning is: evaluate the expression `e`; if its value is the boolean `True` execute `s1`, otherwise execute `s2`.
 - A **while** loop of the form `while (e) {s}`. The operational meaning is: evaluate the expression `e`; if its value is the boolean `False` terminate; otherwise execute `s` once and restart the evaluation of the loop.

The following is an example program in IMP that calculates the factorial of 5.

```
r := 1;
while (0 < n) {
  r := r * n;
  n := n - 1
}
```

Suppose that we initialize the memory so that it maps `n` to 5. After executing this program, the final state of the memory will map `n` to 0, and `r` to 120.

Exercise 1. [Memory]. Download the file corresponding to this assignment from the course campus website. In this first exercise, we will implement a datatype `Mem` to represent the memory. Edit the `Mem.hs` file. It contains the following declaration:

```
data Mem = MkMem [(VariableName, Value)]
```

This declaration means that a memory is represented as a list of pairs, each associating a variable to a value. The list should not contain duplicate variable names. Note that the `MkMem` constructor is not exported by the module, which means that it cannot be used explicitly from other modules. That is, the user can only interact with a memory by means of the operations in the interface.

Complete the definition of the following functions:

- a) `emptyMem :: Mem` — represents an empty memory, in which the value of every variable is 0.
- b) `loadMem :: VariableName -> Mem -> Value` — returns the value associated to the given variable.
- c) `storeMem :: VariableName -> Value -> Mem -> Mem` — assigns a given variable name with a given value.
- d) `showMem :: Mem -> String` — shows the contents of the memory as a string. For example, if the memory maps `a` to 2, and `b` to `True`, and `c` to 10, this function must return `"{a -> 2, b -> True, c -> 10}"`. Variables with value 0 must not be shown as part of the output. Variables must be ordered in alphabetical order of their names.
- e) `eqMem :: Mem -> Mem -> Bool` — returns `True` if and only if the two memories are equal. Two memories are equal if they map the same variables to the same values. Note that it does **not** suffice to check that the underlying lists are equal. For example, `MkMem [("a", VInteger 1), ("b", VInteger 0), ("c", VInteger 2)]` and `MkMem [("c", VInteger 2), ("a", VInteger 1)]` should be equal because both map `a` to 1, `c` to 2, and any other variable to 0.

For example, if we define:

```
test :: Mem
test = storeMem "a" (VInteger 2) (
  storeMem "c" (VInteger 10) (
    storeMem "b" (VBool True) (
      storeMem "a" (VInteger 1) emptyMem)))
```

Then:

```
loadMem "a" test  == VInteger 2
loadMem "b" test  == VBool True
loadMem "c" test  == VInteger 10
loadMem "d" test  == VInteger 0
showMem test      == "{a -> 2, b -> True, c -> 10}"
```

Exercise 2. [Expressions]. In this exercise, we will implement a datatype `Expr` to represent expressions. Edit the `Expression.hs` file. First, this module defines a data types `OpBinary` to represent binary operators (`+`, `*`, `&&`, `||`, `==`, and `<`), and a data type `OpUnary` to represent unary operators (`!` and `-`). Moreover, it defines an `Expr` data type to represent expressions:

```
data Expr = ExprVar VariableName
          | ExprConst Value
          | ExprBinary OpBinary Expr Expr
          | ExprUnary OpUnary Expr
```

For example, the expression `"3 * x + y == - z"` is represented as the following value of type `Expr`:

```
example :: Expr
example = ExprBinary OpEqual
  (ExprBinary OpAdd
    (ExprBinary OpMultiply
      (ExprConst (VInteger 3))
      (ExprVar "x"))
    (ExprVar "y"))
  (ExprUnary OpMinus (ExprVar "z"))
```

- a) Write the type and define a function `foldExpr` that abstracts the structural recursion principle for the `Expr` datatype.
- b) Using the function `foldExpr`, and **without using explicit recursion**, define a function `variablesExpr :: Expr -> [VariableName]` that returns a list without duplicates containing all and only those variables that appear in the expression. For example, if the expression is `"3 * x * z + x == 2 * a"` the list of variables should be `["x", "z", "a"]`.
- c) Using the function `foldExpr`, and **without using explicit recursion**, define a function `renameExpr :: Expr -> VariableName -> VariableName -> Expr` that renames all the occurrences of the first variable in the expression, changing it by the second variable, and vice-versa. For example, if the expression is `"x * x + 2 * y + z"` then renaming `x` by `z` yields the expression `"z * z + 2 * y + x"`.
- d) Using the function `foldExpr`, and **without using explicit recursion**, define a function `evalExpr :: Mem -> Expr -> Value` that returns the value of the expression in the given memory. For example, if the expression is `"3 * x"` and the memory maps `x` to 2, the result should be the value 6. If the expression is `"3 * x == 6"` and the memory maps `x` to 2, the result should be the value `True`.

If the arguments of an expression do not have the correct types, the result must be 0. For example, evaluating `False + 3` yields 0, and evaluating `1 && True` also yields 0.

- e) The file `Expression.hs` contains a test `test0`. Write at least three more tests to validate that `evalExpr` is working correctly. Ensure that all operators are covered in the tests.

Note. In this assignment we are not going to use an automated testing tool¹, but please include the full code for the tests, including the expected values, in such a way that the result of the test is `True` if and only if it succeeds.

Exercise 3. [First interpreter: basic execution]. Edit the file `Interpreter1.hs`. It defines a `Stmt` data type to represent statements:

```
data Stmt = StmtSkip
          | StmtAssign VariableName Expr
          | StmtSequence Stmt Stmt
          | StmtIf Expr Stmt Stmt
          | StmtWhile Expr Stmt
```

Remember that `StmtSkip` is a program that does nothing, that is, executing `StmtSkip` does not produce any change in the memory. For example the following IMP program:

```
r := 1;
while (0 < n) {
  r := r * n;
  n := n + -1
}
```

is represented as the following value of type `Stmt`:

```
factorial :: Stmt
factorial =
  StmtSequence
    (StmtAssign "r" (ExprConst (VInteger 1)))
    (StmtWhile
      -- Condition:
      (ExprBinary OpLessThan (ExprConst (VInteger 0)) (ExprVar "n"))
      -- Body:
      (StmtSequence
        (StmtAssign "r" (ExprBinary OpMultiply (ExprVar "r") (ExprVar "n")))
        (StmtAssign "n" (ExprBinary OpAdd (ExprVar "n") (ExprConst (VInteger (-1)))))))
```

¹There are many automated testing tools for Haskell, such as `HUnit`.

- a) Define a function `executeStmt :: Stmt -> Mem -> Mem` that receives a statement and a memory, executes the statement and returns the final value of the memory. For example, if `mem` is the memory that maps `n` to 5, then `execute factorial mem` returns a memory that maps `r` to 120.
- b) Define a value `twoTwoThePowerOfN :: Stmt` that represents a program in IMP that receives a variable `n` as an input, which must be a non-negative integer, and returns 2^n in the variable `r`. You can follow the example code above. For example, if `mem` is the memory that maps `n` to 7, then `execute twoTwoThePowerOfN mem` returns a memory that maps `r` to 128.
- c) The file `Interpreter1.hs` contains a test `test0`. Write at least three more tests to validate that `executeStmt` is working correctly. Ensure to cover all the forms of statements, including the if-then-else statement that has not been covered before.

Exercise 4. [Second interpreter: writing values in an output log]. In this exercise we extend the IMP language. Besides the memory, the runtime semantics of IMP now includes a **output log**, which is a list of values. The syntax of statements is extended with a statement `write(e)` that writes a value in the output log. The log can only be *extended*, that is, one can append values at the end of the log. The program cannot read the log, and it cannot erase or change values that have already been written in the log. For example, if initial the memory maps `n` to 10, then the following program:

```
i := 0;
while (i < n) {
  write(i);
  i := i + 1
}
```

produces the output log `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`.

- a) Edit the file `Interpreter2.hs`. In this exercise, we use the following type synonym, which is already included in this file:

```
type OutputLog = [Value]
```

Define a function `executeStmtWithLog :: Stmt -> Mem -> (OutputLog, Mem)` that receives a statement and a memory, executes the statement and returns a pair (ℓ, m) where ℓ is the final output log and m is the final value of the memory.

- b) Define a value `countToN :: Stmt` that represents a program in IMP that receives a variable `n` as an input, which must be a non-negative integer, and counts up to `n`, writing all the integers `0, 1, ..., n - 1` in the output log.
- c) The file `Interpreter2.hs` contains a test `test0`. Write at least three more tests to validate that `executeStmtWithLog` is working correctly. You should test that a sequence statement returns the log in left-to-right order, and that an if-then-else statement produces output for exactly one of the two branches.

Exercise 5. [Third interpreter: early termination and returning values]. In this exercise we extend the IMP language in a different direction. Now there isn't an output log, but programs may **return a value**. The syntax of statements is extended with a statement `return(e)` that **immediately halts the program**, returning the value represented by the expression `e`. For example, if initial the memory maps `n` to 16, then the following program calculates its square root, and terminates returning 4:

```
i := 0;
while (true) {
  if (!(i * i < n)) {
    return(i)
  } else {
    i := i + 1
  }
}
```

- a) Edit the file `Interpreter3.hs`. In this exercise, we use the following auxiliary data type synonym, which is already included in this file:

```
data Result = RContinue Mem
            | RReturn Value
```

Define a function `executeStmtWithReturn :: Stmt -> Mem -> Result` that receives a statement and a memory, executes the statement and returns an instance of the `Result` datatype. The result should be interpreted as follows. First, if the program has not executed a `return` statement, the result should be of the form `RContinue m`, where m is the final value of the memory. Second, if the program has not executed a `return` statement, the result should be of the form `RReturn v`, where v is the returned value.

- b) Define a value `squareRoot :: Stmt` that represents a program in IMP that receives a variable n as an input, which must be a non-negative integer, and returns the square root of n . If n is not a perfect square, it should return the first i such that $i * i > n$. You can follow the example code above. For example, if `mem` is the memory that maps `n` to 99, then `execute twoTwoThePowerOfN mem` should be the result `RReturn 10`.
- c) The file `Interpreter3.hs` contains a test `test0`. Write at least three more tests in order to validate that `executeStmtWithReturn` is working correctly. For a sequence statement `s1; s2` you should ensure that if `s1` returns a value then `s2` is not executed, while if `s1` does not return a value then `s2` must be executed and it can return a value. For an if-then-else statement `if (e) {s1} else {s2}`, you should ensure that if the condition is true then the return statements in `s2` have no effect. Symmetrically, the return statements in `s1` should have no effect if the condition is false.

Submission

- You must submit your solution using the moodle website.
- Your solution must be submitted as a `zip` file containing all the `*.hs` files.
- The deadline for submission is **Thursday 2026-04-30 until 23:59:59**, Shantou time (GMT+8).
- **Late submissions** will be accepted until Friday 2026-05-01 23:59:59 with **25% penalty**.
- No submissions will be accepted after Friday 2026-05-01 23:59:59.
- Sometimes the course moodle experiences delays. Please **do not wait to submit until the last minute**.